

CityTech TTP Summer Bootcamp

Overfitting and Honest Scoring

2026-07-20

Agenda

1. Review
2. Follow up: irrelevant features and overfitting
3. Keep exploring last class's notebook
4. Why we hold rows back
5. `train_test_split` in code
6. Train, validation, and test
7. Demo: train and test in a notebook
8. Project work time

Review

What are the most important things you remember from last week?

Follow Up: Does Adding Irrelevant Features Affect the Model?

Yes, though not in the way the score suggests.

- Adding a feature **never lowers** R-squared, even a column of pure random noise
- Each new feature is another **knob** the model can turn to chase a fake pattern
- The score rises while the model gets **worse** at anything new: this is **overfitting**
- Give it as many knobs as rows and it fits **every point**: a perfect score, nothing learned

A perfect R-squared is a warning, not an achievement.

Keep Exploring Last Class's Notebook

Go back to `demos/linear-regression-intro/` and push on it.

- Re-run it, then **change** things: swap features, predict a different apartment
- Re-read the **correlation heatmap**: which columns earn their place?

Comfort with the workflow matters more than memorizing the API.

Hold Some Rows Back

If the score only ever rises, how do we tell a real model from a memorizer?

- You would not test a student on the exact questions they studied
- Scoring a model on the rows it trained on is an **open book exam**
- So **split** the data before training: keep most of it, hold the rest back
- **Train** on the rows you kept; **test** on the rows the model never saw

Predicting rent from `size_sqft` alone: train R-squared **0.769**, test R-squared **0.703**.

train_test_split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)           # hold back 20% of the rows

model = LinearRegression().fit(X_train, y_train)    # learn here
model.score(X_test, y_test)                        # grade there
```

- `test_size=0.2` : keep **20 percent** of the rows back for testing
- `random_state=0` : the same split every run, so results are **reproducible**
- Never let the model see `X_test` until you score

Fit on **train**. Report the **test** number.

A Third Set: Validation

Some workflows split the data **three** ways, not two.

- **Train:** where the model learns
- **Validation:** where you compare options and tune
- **Test:** the final score, looked at **once**

You **fit** to the train rows, and you **tune** until validation looks good, so both flatter you by construction. Only the **test** rows can still come out badly, because you never touched them.

The only useful test is one you might fail.

Demo: Train and Test

Open `demos/train-test-split/` to run all of this yourself.

- Scores a model on its **training** rows, then honestly on a **test** set
- Shows what `test_size` and `random_state` actually do
- Plots which rows the model **learned from** and which it was **graded on**
- Same NYC rent data as last class

```
cd demos/train-test-split  
jupyter lab
```

Keep Going on Your Project

Data for a Better NYC: at least **two datasets**, one from **NYC Open Data**. Presentations are tomorrow.

- Get your datasets actually **working together**: merged, compared, or one giving context
- Finish the **visualizations** that carry your findings
- Write the story: what did you find, and what should the city do about it?
- Commit from **both** members

Use today's time, and bring your questions.

Today's Exit Ticket

- [] Explain what **overfitting** is, and how irrelevant features cause it
- [] Explain why a **perfect R-squared** is a warning sign
- [] **Split** data into train and test, and say which score you report
- [] Use `train_test_split` with `test_size` and `random_state`
- [] Say in one sentence what a **validation** set is for
- [] Re-run last class's notebook with a change of your own